

YingMusic-Singer: Controllable Singing Voice Synthesis with Flexible Lyric Manipulation and Annotation-free Melody Guidance

Chunbo Hao^{1,2}, Junjie Zheng², Guobin Ma¹, Yuepeng Jiang¹, Huakang Chen¹, Wenjie Tian¹,
Gongyu Chen², Zihao Chen², Lei Xie^{1,**}

¹ Audio, Speech and Language Processing Group (ASLP@NPU)
School of Computer Science, Northwestern Polytechnical University, China
² AI Lab, Giant Network, China

cbhao@mail.nwpu.edu.cn, zhengjunjie@ztgame.com, lxie@nwpu.edu.cn

Abstract

Regenerating singing voices with altered lyrics while preserving melody consistency remains challenging, as existing methods either offer limited controllability or require laborious manual alignment. We propose YingMusic-Singer, a fully diffusion-based model enabling melody-controllable singing voice synthesis with flexible lyric manipulation. The model takes three inputs: an optional timbre reference, a melody-providing singing clip, and modified lyrics, without manual alignment. Trained with curriculum learning and Group Relative Policy Optimization, YingMusic-Singer achieves stronger melody preservation and lyric adherence than Vevo2, the most comparable baseline supporting melody control without manual alignment. We also introduce LyricEditBench, the first benchmark for melody-preserving lyric modification evaluation. The code, weights, benchmark, and demos are publicly available at <https://github.com/ASLP-lab/YingMusic-Singer-Plus>.

Index Terms: singing voice synthesis, lyric editing, reinforcement learning, diffusion model

1. Introduction

Singing Voice Synthesis (SVS) aims to generate human-like singing voices from musical scores, lyrics, and timbre references. Modern systems [1, 2, 3, 4, 5, 6] achieve high-fidelity synthesis, yet most rely on precisely annotated paired data associating each phoneme with an exact pitch contour and duration. While such fine-grained control is indispensable for professional music production, preparing these annotations creates a prohibitive barrier for an increasingly important use case, *singing voice editing*, which regenerates an existing singing voice with modified lyrics while preserving the original melodic and rhythmic structure. This capability is highly desirable for song adaptation, personalized cover generation, rapid prototyping of vocal arrangements, and multilingual song localization.

Existing editing paradigms fall into two categories. The first adopts an in-context learning strategy that masks the region to be edited and regenerates it conditioned on the surrounding context and target lyrics [7, 8]. Although convenient, this approach is restricted to local segments and provides limited melody control. The second category, widely adopted in practice, relies on commercial SVS models such as Synthesizer V¹ and ACE Studio², where users first need to transcribe the origi-

nal vocal into MIDI, then assign the modified lyrics to the notes and durations, and re-synthesize the audio. This pipeline offers strong controllability but demands manual effort, and the complexity increases for tasks such as cross-lingual translation. In summary, existing approaches either depend on the surrounding context to recover melody or require manual alignment of word-level timestamps with melody information, limiting their flexibility and scalability. Several recent efforts address these challenges. Vevo2 [9] achieves melody-controllable generation, yet suffers from reduced intelligibility and poor melody adherence. SoulX-Singer [10] supports existing singing as melody input but still requires manual alignment of word-level timestamps, leaving the core burden unresolved. While these recent efforts narrow the gap, they either achieve suboptimal performance or still need manual alignment.

To address these challenges, we propose *YingMusic-Singer*, a fully diffusion-based model for melody-preserving lyric editing. First, YingMusic-Singer introduces a streamlined editing paradigm that synthesizes singing voices from only three inputs, namely an optional timbre reference clip, a singing clip providing the target melody, and the corresponding modified lyrics, without the need for manual alignment or precise annotation. Second, to mitigate limited phoneme generalization caused by the small scale and challenging vocal techniques in singing data, and to address the inherent trade-off between faithful lyric reproduction and melody adherence, we employ a curriculum training strategy combined with Group Relative Policy Optimization (GRPO) [11, 12], enabling strong performance on both dimensions simultaneously. Third, we construct *LyricEditBench* based on GTSinger [13], the first benchmark for lyric modification evaluation under matched melody conditions, covering six common editing scenarios with balanced sampling for fair and comprehensive comparison. Experiments show that YingMusic-Singer outperforms Vevo2 [9], currently a comparable alignment-free alternative, in both melody preservation and lyric adherence. We publicly release the model weights, inference code, and LyricEditBench to facilitate further research and advance the practical adoption of singing voice editing.

2. Methodology

2.1. Architecture Overview

As shown in Figure 1, YingMusic-Singer generates singing voices at 44.1 kHz from three inputs: an optional timbre reference, a melody-providing singing clip, and corresponding modified lyrics. It comprises: (1) a Variational Autoencoder (VAE) following Stable Audio 2 [14], whose encoder $\mathcal{E}$ down-

^{**}indicates the corresponding author.

¹<https://dreamtonics.com/synthesizerv>

²<https://acestudio.ai>

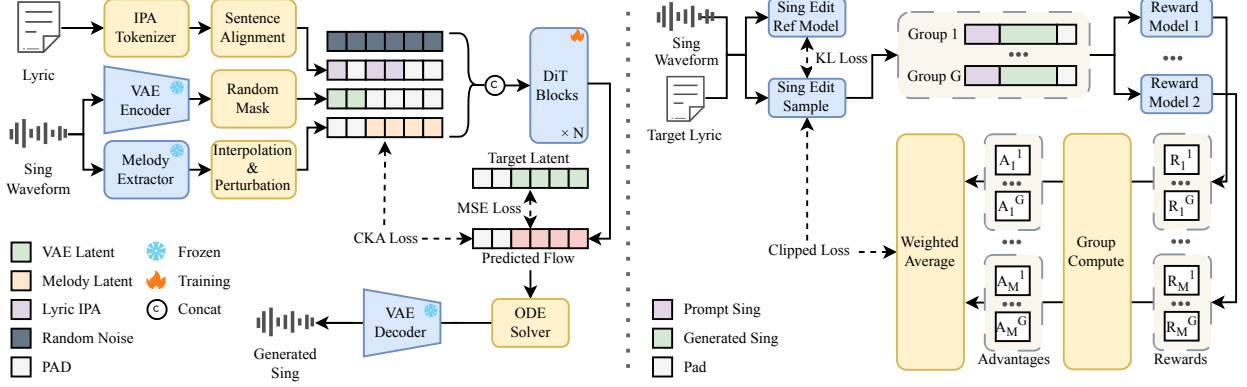


Figure 1: Overall architecture of YingMusic-Singer. Left: the training pipeline consisting of a Variational Autoencoder, a Melody Extractor, an IPA Tokenizer, and DiT-based conditional flow matching. Right: the GRPO training pipeline.

samples a stereo 44.1 kHz singing waveform $\mathbf{x} \in \mathbb{R}^{T \times 2}$ by a factor of 2048 into $\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{T' \times D}$, and whose decoder $\mathcal{D}$ reconstructs high-fidelity audio $\hat{\mathbf{x}} = \mathcal{D}(\mathbf{z})$ at inference; (2) a Melody Extractor built upon the encoder of a pre-trained MIDI extraction model, whose intermediate representations naturally capture disentangled melody information. It produces $\mathbf{h} = \mathcal{M}(\mathbf{M}) \in \mathbb{R}^{L \times D_m}$, which is then temporally interpolated to $\tilde{\mathbf{h}} \in \mathbb{R}^{T' \times D_m}$ to match the VAE latent frame rate; (3) an IPA Tokenizer that converts both Chinese and English lyrics into a unified discrete phoneme sequence. To ensure the model correctly distinguishes between prompt and generation regions, avoiding phoneme omission or repetition at boundaries, we adopt sentence-level alignment following DiffRhythm [15]. Each lyric sentence is converted into an IPA subsequence and placed at its corresponding onset frame within a padded frame-level sequence of length T' . The aligned sequence is passed through a learnable embedding layer to yield $\mathbf{e} \in \mathbb{R}^{T' \times D_e}$. During inference, prompt lyrics are placed at the beginning of the sequence and target lyrics at the start of the masked region, requiring no timestamp annotation from the user; and (4) a **DiT-based CFM backbone** following F5-TTS [16].

During training, a proportion γ of VAE latent frames is randomly masked as the synthesis target, with the unmasked portion serving as timbre context. Let $\mathbf{z}_{\text{ctx}}$ denote the unmasked VAE latent (zero-filled in masked regions). The condition $\mathbf{c} = [\tilde{\mathbf{h}}; \mathbf{e}; \mathbf{z}_{\text{ctx}}]$ is concatenated with the noisy latent $\mathbf{z}_t = (1-t)\mathbf{z}_0 + t\mathbf{z}_1$ along the channel dimension and fed into the CFM, which learns a velocity field via:

$$\mathcal{L}_{\text{MSE}} = \mathbb{E}_{t, \mathbf{z}_0, \mathbf{z}_1, \mathbf{c}} \|\mathbf{v}_\theta(\mathbf{z}_t, t, \mathbf{c}) - (\mathbf{z}_1 - \mathbf{z}_0)\|^2. \quad (1)$$

2.2. Curriculum Training

To mitigate limited phoneme generalization caused by the small scale and challenging vocal techniques in singing data, YingMusic-Singer first undergoes *TTS Pretraining* without melody conditioning. The subsequent *Singing Voice Supervised Fine-Tuning (SFT)* stage has two phases. Phase 1 enables sentence-level alignment on singing data, allowing the model to adapt to the singing domain. Phase 2 activates melody conditioning and introduces a Centered Kernel Alignment (CKA) loss to enforce melody adherence. Given the predicted $\mathbf{v}_\theta$ and melody $\tilde{\mathbf{h}}$, CKA measures their alignment via Gram matrices $\mathbf{K} = \mathbf{v}_\theta \mathbf{v}_\theta^\top$ and $\mathbf{L} = \tilde{\mathbf{h}} \tilde{\mathbf{h}}^\top$:

$$\mathcal{L}_{\text{CKA}} = 1 - \frac{\|\mathbf{K}^\top \mathbf{L}\|_F^2}{\|\mathbf{K}^\top \mathbf{K}\|_F \|\mathbf{L}^\top \mathbf{L}\|_F}, \quad (2)$$

and the total phase 2 loss is $\mathcal{L}_{\text{SFT}} = \mathcal{L}_{\text{MSE}} + \lambda \mathcal{L}_{\text{CKA}}$.

2.3. Group Relative Policy Optimization

While curriculum training achieves high performance, SFT Phase 2 simultaneously degrades PER, exposing a persistent trade-off. $\mathcal{L}_{\text{MSE}}$ targets holistic latent reconstruction and cannot isolate specific deficiencies for targeted optimization, while adjusting λ in $\mathcal{L}_{\text{CKA}}$ only shifts the balance without resolving the trade-off itself. Moreover, inevitable noise in large-scale singing data, such as backing vocals and quality artifacts, caps model performance at the dataset ceiling. To overcome these limitations, reinforcement learning (RL) becomes essential. PPO requires a value network that is expensive to train. Offline methods such as DPO suffer from distribution shift, as pre-collected preference data becomes stale when the policy improves. GRPO [11, 12] operates online and estimates baselines from within-group reward statistics, eliminating the value network while remaining efficient and stable.

Following recent efforts [17, 18, 19], we convert the deterministic ODE trajectory into an SDE but restrict stochastic steps to a bounded window, ensuring precise advantage attribution to exploratory steps. To prevent collapse toward a single reward dimension, we employ M reward models jointly and compute the advantage for each sample as

$$A^i = \sum_{k=1}^M w_k \frac{R_k^i - \text{mean}(\{R_k^j\})}{\text{std}(\{R_k^j\})}, \quad (3)$$

and the final GRPO loss is

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|S|} \sum_{t \in S} (-\mathcal{L}_{\text{clip}} + \beta D_{\text{KL}}), \quad (4)$$

where $\mathcal{L}_{\text{clip}}$ is the clipped surrogate objective over the current-to-old policy likelihood ratio, D_{KL} regularizes the current policy toward the reference, G is the group size, S the set of SDE sampling steps, and β the KL regularization strength.

2.4. LyricEditBench

We build LyricEditBench from GTSinger [13] by removing all Paired Speech Group content, deduplicating audio via MD5 hashing, and excluding clips exceeding 15 seconds. DeepSeek V3.2 [20] then generates modified lyrics for each of the six modification types in Table 1. Given original lyrics and modification instructions, the LLM produces revised versions, with

Table 1: Task types for lyric modification in LyricEditBench.

Abbr.	Task Type	Description
PSub	Partial Substitution	Substitute part of the words
FSub	Full Substitution	Completely rewrite the song
Del	Deletion	Delete some words
Ins	Insertion	Insert some words
Trans	Translation	CN $\leftrightarrow$ EN translation
Mix	Code-Mixing	Mixed-language lyrics

non-compliant outputs discarded, yielding 11,535 valid samples. Samples are classified by singer gender (male/female) and language (Chinese/English) into four categories, then organized by modification type. For each combination, we select 30 samples per singing technique (covering the six techniques in GTSinger) and 120 for the technique-free category, resulting in 300 per modification type per category and 7,200 test instances in total. For each instance, a timbre prompt of no more than 15 seconds is randomly drawn from the remaining audio pool, so each LyricEditBench instance comprises a melody reference clip, a timbre prompt, and the corresponding modified lyrics.

3. Experimental Setup

Dataset.

The Chinese and English subsets of Emilia [21] are used for TTS pretraining. For Singing Voice SFT, internally licensed music tracks are processed by SongFormer [22] to segment structural boundaries and label function categories, discarding non-vocal segments. Vocal stems are then isolated using Mel-band RoFormer [23]. We retain clips between 2 and 30 seconds, splitting longer ones at sentence boundaries, ultimately obtaining 33,562.6 hours of singing data. The GRPO dataset is constructed by filtering SFT data with three criteria: ASR transcript verification, retaining only clips with a word error rate below 5%, speaker diarization via pyannote [24, 25], keeping only single-speaker clips, and a DNSMOS P.808 quality score [26, 27] threshold of 3.5. This yields approximately 20,240 curated clips with balanced Chinese and English content. The test set in the proposed LyricEditBench is strictly excluded from training.

Evaluation Metrics. We evaluate models with four objective and two subjective metrics. For objective evaluation, *Phoneme Error Rate (PER)* measures phoneme-level intelligibility, as singing exhibits lower semantic density, fewer contextual cues, and greater pronunciation variation than speech. Both Chinese and English clips are transcribed by singing-trained Qwen3-ASR-1.7B [28] and converted to phoneme sequences with tone markers removed. *Speaker Similarity (SIM)* follows F5-TTS [16], extracting speaker embeddings with a WavLM-large-based verification model and computing cosine similarity. *F0 Pearson Correlation (F0-CORR)* measures melody adherence via frame-wise Pearson correlation between F0 contours of generated and reference clips using RMVPE [29]. *Vocal Score (VS)* adopts VocalVerse2 [30] as a learned metric aligned with human perceptual preferences. For subjective evaluation, 120 samples uniformly sampled across task types and languages are rated by 30 listeners on two dimensions: *Naturalness Mean Opinion Score (N-MOS)* for overall perceptual quality and naturalness, and *Melody Mean Opinion Score (M-MOS)* for faithfulness to the reference melody, both on a 5-point scale.

Implementation Details. We adopt the VAE from Stable Audio 2 [14] ($D = 64$), the encoder of SOME³ as Melody

Extractor ($D_m = 128$, temporal dropout 0.1), and a DiT backbone following F5-TTS [16] (22 layers, 16 heads, hidden dim 1024, $D_e = 512$). The full system has ~ 727.3 M parameters (453.6M CFM, 156.1M VAE, 117.6M Melody Extractor), trained on $8 \times A800$ 80GB GPUs with DDP and bf16. Across all stages, 70%–100% of latent frames are randomly masked. TTS pretraining runs for 1M steps (batch duration 1.268h, lr $1e-4$). Singing Voice SFT Phase 1 disables melody conditioning for 240K steps; Phase 2 enables it for 170K steps (λ decayed from 0.3 to 0.01 over the first 2K steps; batch duration 1.69h, lr $2.5e-5$). For GRPO, $G=8$ candidates are scored by $M=4$ equally weighted reward models (SDE noise $a=0.8$, window $\mathcal{W}$ with $w_{\min}=1$, $w_s=8$, $\epsilon_u=0.01$, $\epsilon_l=0.002$, $\beta=1$), optimized for 1.2K steps (batch size 6, lr $7e-6$) without CFG. Inference uses 32 ODE steps with CFG scale 3.

4. Experimental Results

4.1. Main Results

We compare against Vevo2 [9], a token-based autoregressive model with disentangled timbre and melody control. For Sing Edit (self-timbre), the timbre and melody references share the same clip; for Melody Control (cross-timbre), the timbre reference is replaced accordingly. Vevo2 is the most direct baseline, as it supports the same functionalities under comparable input conditions. Other recent systems operate under fundamentally different paradigms. In-context learning approaches require surrounding audio context with manually aligned edit boundaries, restricting them to local segment editing. SoulX-Singer relies on precise character-level timestamps; obtaining such annotations via ASR introduces non-trivial character-count mismatches, while ground-truth timestamps from curated datasets would grant an unrealistic advantage absent in practice.

As shown in Table 2, YingMusic-Singer consistently outperforms Vevo2 across all six modification types under both Melody Control and Sing Edit settings in PER, F0-CORR, and VS, demonstrating strong adherence to both the reference melody and modified lyrics. The intelligibility gap is most pronounced on Trans and Mix tasks, indicating that reconstructing a substantially different phoneme sequence while preserving melody is inherently challenging. Note that PER on Mix tasks may be further inflated by ASR hallucinations on code-switched utterances. Vevo2’s incomplete melody disentanglement tends to reduce intelligibility and introduce hallucinations, whereas YingMusic-Singer’s unified IPA tokenization and GRPO-based lyric adherence optimization maintain robustness even under these extreme conditions. F0-CORR further distinguishes the two systems: benefiting from CKA alignment and GRPO, YingMusic-Singer maintains consistently high correlation across all tasks and languages, whereas Vevo2 fluctuates considerably, suggesting less robust melody control. For SIM, Vevo2 benefits from its multi-stage architecture, where an autoregressive LLM handles melody and content generation while a dedicated CFM focuses on timbre reconstruction, effectively easing speaker modeling. YingMusic-Singer instead adopts a single-stage CFM that jointly models all factors, prioritizing architectural simplicity for practical deployment. However, the higher SIM of Vevo2 comes at the cost of degraded PER and F0-CORR. In practice, faithfully rendering modified lyrics while preserving the original melodic structure remains the primary concern in lyric editing. As shown in Table 3, YingMusic-Singer consistently achieves higher N-MOS and M-MOS than Vevo2 across both tasks and languages. The strong

³<https://github.com/openvpi/SOME>

Table 2: Comparison with Baseline Model on LyricEditBench across Task Types in Table 1 and Languages. Metrics (M): P: PER, S: SIM, F: F0-CORR, V: VS are detailed in Section 3. Best results are in **bold**.

Task	Model	M	Chinese						English					
			PSub	FSub	Del	Ins	Trans	Mix	PSub	FSub	Del	Ins	Trans	Mix
Melody Control (cross-timbre)	Vevo2 [9]	P ↓	0.1378	0.1462	0.1545	0.1872	0.4409	0.4757	0.3352	0.3481	0.3812	0.3135	0.8019	0.5132
		S ↑	0.6462	0.6550	0.6457	0.6551	0.6115	0.6490	0.6357	0.6161	0.6277	0.6359	0.6237	0.6325
		F ↑	0.8471	0.8188	0.8345	0.8552	0.7678	0.8526	0.8794	0.8409	0.8924	0.8888	0.8776	0.8927
		V ↑	1.3578	1.3784	1.3491	1.1346	1.3061	1.4208	1.0340	1.1217	1.0476	0.9610	1.0281	1.0925
	Ours	P ↓	0.0192	0.0197	0.0458	0.0208	0.0881	0.1563	0.0685	0.0692	0.1053	0.0716	0.0413	0.2668
		S ↑	0.6543	0.6561	0.6489	0.6552	0.5791	0.6395	0.6078	0.5914	0.6001	0.5889	0.5982	0.6076
		F ↑	0.9364	0.9428	0.9351	0.9381	0.9378	0.9352	0.9355	0.9279	0.9309	0.9315	0.9290	0.9389
		V ↑	2.0779	2.1419	2.1219	1.9887	1.9372	2.1002	1.5054	1.5418	1.6081	1.4036	1.5769	1.5060
Sing Edit (self-timbre)	Vevo2 [9]	P ↓	0.1290	0.1303	0.1596	0.1810	0.4111	0.4659	0.3414	0.3538	0.3531	0.2944	0.7680	0.4951
		S ↑	0.7875	0.7729	0.8269	0.8324	0.7252	0.8015	0.7971	0.7729	0.8183	0.8378	0.7563	0.8346
		F ↑	0.8858	0.8805	0.8969	0.9115	0.8456	0.9023	0.9258	0.9278	0.9365	0.9415	0.9137	0.9465
		V ↑	1.4860	1.5100	1.5094	1.2935	1.3535	1.4377	1.0910	1.1110	1.1682	1.1156	1.1178	1.1453
	Ours	P ↓	0.0214	0.0186	0.0946	0.0426	0.1009	0.1903	0.0906	0.0782	0.1700	0.1070	0.0538	0.2946
		S ↑	0.7622	0.7392	0.7874	0.8028	0.6539	0.7564	0.7398	0.7105	0.7764	0.7714	0.6918	0.7708
		F ↑	0.9615	0.9587	0.9628	0.9642	0.9542	0.9607	0.9610	0.9563	0.9675	0.9660	0.9498	0.9668
		V ↑	1.9761	2.0345	1.8837	1.7689	1.9371	1.9283	1.4448	1.4086	1.3820	1.2553	1.4788	1.3464

Table 3: Subjective evaluation on LyricEditBench. N-MOS and M-MOS denote naturalness and melody adherence, respectively.

Task	Model	ZH		EN	
		N↑	M↑	N↑	M↑
Melody Control	Vevo2 [9]	4.25±0.06	4.28±0.05	4.31±0.05	4.31±0.04
	Ours	4.31±0.04	4.44±0.04	4.36±0.05	4.51±0.03
Sing Edit	Vevo2 [9]	4.48±0.05	4.41±0.05	4.44±0.04	4.50±0.04
	Ours	4.52±0.04	4.55±0.04	4.55±0.04	4.58±0.03

subjective scores also indicate that GRPO optimization does not overfit to the reward models but generalizes to human perception. Vevo2 receives lower ratings overall with notably higher variance, and listeners report perceptible artifacts such as unfaithful lyric rendering and melodic misalignment in its outputs, suggesting less robust generation quality.

Beyond model comparison, the breadth of evaluation across six editing types, two languages, and both objective and subjective metrics further validates LyricEditBench as a comprehensive benchmark for melody-preserving lyric editing, supporting future research on song adaptation, cover generation, and cross-lingual vocal arrangement.

4.2. Ablation Study

As shown in Table 4, the curriculum learning pipeline introduces clearly separable improvements at each stage. TTS Pre-train establishes articulatory priors but lacks singing capability (F0-CORR near zero), with PER degrading significantly when a singing clip serves as the ICL prompt, indicating poor domain adaptation. SFT Phase 1 substantially improves all metrics, achieving the lowest PER as the model adapts to the singing domain while freely generating melody from the ICL prompt, bypassing the demanding task of explicit melody alignment. F0-CORR under Sing Edit improves slightly, showing partial melody capture from context alone, yet explicit guidance remains necessary for faithful reproduction. SFT Phase 2 activates the Melody Extractor and raises F0-CORR above 0.92, though PER increases, reflecting the difficulty of jointly main-

Table 4: Ablation Study on LyricEditBench. Best results are **bold**, second best underlined.

Lg	Variant	Melody Control				Sing Edit			
		P ↓	S ↑	F ↑	V ↑	P ↓	S ↑	F ↑	V ↑
ZH	TTS Pretrain	0.41	0.57	0.01	0.50	0.37	0.59	0.06	0.49
	SFT Phase1	0.05	0.68	0.03	1.55	0.05	0.73	0.31	1.53
	SFT Phase2	0.08	0.63	<u>0.92</u>	<u>1.57</u>	0.11	<u>0.75</u>	<u>0.95</u>	<u>1.62</u>
	-w/o CKA	0.08	<u>0.64</u>	<u>0.91</u>	<u>1.57</u>	0.12	<u>0.75</u>	<u>0.93</u>	1.61
	-w/o Dist	0.45	0.63	0.94	1.42	0.46	0.79	<u>0.95</u>	1.55
	Full Model	<u>0.06</u>	<u>0.64</u>	0.94	2.06	<u>0.08</u>	<u>0.75</u>	0.96	1.92
EN	TTS Pretrain	0.46	0.54	0.01	0.49	0.43	0.56	0.00	0.50
	SFT Phase1	0.10	0.65	0.03	1.07	0.11	0.73	0.40	1.13
	SFT Phase2	<u>0.14</u>	<u>0.60</u>	0.92	1.17	0.19	<u>0.75</u>	0.96	<u>1.21</u>
	-w/o CKA	<u>0.14</u>	<u>0.60</u>	0.90	<u>1.19</u>	0.20	<u>0.75</u>	<u>0.94</u>	1.18
	-w/o Dist	0.48	0.58	0.95	1.00	0.49	0.78	0.96	0.98
	Full Model	0.10	<u>0.60</u>	<u>0.93</u>	1.52	<u>0.13</u>	0.74	0.96	1.39

taining melody fidelity and lyric faithfulness. GRPO resolves this trade-off by recovering PER while further improving F0-CORR and VS with SIM unchanged, confirming that reward-based optimization jointly enhances all dimensions.

In SFT Phase 2, incorporating CKA further improves melody adherence, as reflected in F0-CORR gains. For the perturbation ablation (w/o Dist), we remove the temporal dropout applied to the melody latent $\mathbf{h}$. This causes severe intelligibility degradation, as the unperturbed latent retains residual semantic information that the model exploits to bypass genuine lyric generation. Temporal dropout eliminates this leakage, forcing reliance on the abstract melodic contour and preserving prosodic structure while allowing free generation of modified lyrics.

5. Conclusion

We present YingMusic-Singer, a melody-controllable singing voice editing model that synthesizes from a timbre reference, a melody-providing singing clip, and modified lyrics without manual alignment. Through curriculum training and GRPO-based reinforcement learning, YingMusic-Singer achieves superior melody preservation and lyric adherence on LyricEditBench, the first comprehensive benchmark we introduce for this task, demonstrating strong potential for practical end-to-

end singing voice editing.

6. Generative AI Use Disclosure

Generative AI tools are used solely for linguistic refinement and play no role in methodology, experimentation, interpretation, or the production of scientific results. The authors bear full intellectual responsibility for all content in this manuscript.

7. References

- [1] J. Liu, C. Li, Y. Ren, F. Chen, and Z. Zhao, “Diffsinger: Singing voice synthesis via shallow diffusion mechanism,” in *AAAI*. AAAI Press, 2022, pp. 11 020–11 028.
- [2] J. He, J. Liu, Z. Ye, R. Huang, C. Cui, H. Liu, and Z. Zhao, “Rmssinger: Realistic-music-score based singing voice synthesis,” in *ACL (Findings)*, ser. Findings of ACL, vol. ACL 2023. Association for Computational Linguistics, 2023, pp. 236–248.
- [3] Y. Zhang, R. Huang, R. Li, J. He, Y. Xia, F. Chen, X. Duan, B. Huai, and Z. Zhao, “Stylesinger: Style transfer for out-of-domain singing voice synthesis,” in *AAAI*. AAAI Press, 2024, pp. 19 597–19 605.
- [4] F. Wang, B. Bai, Y. Deng, J. Xue, Y. Gao, and Y. Li, “Expressivesinger: Synthesizing expressive singing voice as an instrument,” in *ISCSLP*. IEEE, 2024, pp. 304–308.
- [5] Y. Yu, J. Shi, Y. Wu, Y. Tang, and S. Watanabe, “Visinger2+: End-to-end singing voice synthesis augmented by self-supervised learning representation,” in *SLT*. IEEE, 2024, pp. 719–726.
- [6] Y. Zhang, W. Guo, C. Pan, D. Yao, Z. Zhu, Z. Jiang, Y. Wang, T. Jin, and Z. Zhao, “Tcsinger 2: Customizable multilingual zero-shot singing voice synthesis,” in *ACL (Findings)*, ser. Findings of ACL, vol. ACL 2025. Association for Computational Linguistics, 2025, pp. 13 280–13 294.
- [7] S. Lei, Y. Zhou, B. Tang, M. W. Y. Lam, F. Liu, H. Liu, J. Wu, S. Kang, Z. Wu, and H. Meng, “Songcreator: Lyrics-based universal song generation,” in *NeurIPS*, 2024.
- [8] C. Yang, S. Wang, H. Chen, J. Yu, W. Tan, R. Gu, Y. Xu, Y. Zhou, H. Zhu, and H. Li, “Songeditor: Adapting zero-shot song generation language model as a multi-task editor,” in *AAAI*. AAAI Press, 2025, pp. 25 597–25 605.
- [9] X. Zhang, J. Zhang, Y. Wang, C. Wang, Y. Chen, D. Jia, Z. Chen, and Z. Wu, “Vevo2: A unified and controllable framework for speech and singing voice generation,” *CoRR*, vol. abs/2508.16332, 2025.
- [10] J. Qian, H. Meng, T. Zheng, P. Zhu, H. Lin, Y. Dai, H. Xie, W. Cao, R. Shang, J. Wu, H. Liu, H. Wen, J. Zhao, Z. Jiang, Y. Chen, S. Yin, M. Tao, J. Wei, L. Xie, and X. Wang, “Soulxsinger: Towards high-quality zero-shot singing voice synthesis,” *CoRR*, vol. abs/2602.07803, 2026.
- [11] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” *CoRR*, vol. abs/2402.03300, 2024.
- [12] D. Guo, D. Yang, H. Zhang, J. Song *et al.*, “Deepseek-r1 incentivizes reasoning in llms through reinforcement learning,” *Nat.*, vol. 645, no. 8081, pp. 633–638, 2025.
- [13] Y. Zhang, C. Pan, W. Guo, R. Li, Z. Zhu, J. Wang, W. Xu, J. Lu, Z. Hong, C. Wang, L. Zhang, J. He, Z. Jiang, Y. Chen, C. Yang, J. Zhou, X. Cheng, and Z. Zhao, “Gtsinger: A global multi-technique singing corpus with realistic music scores for all singing tasks,” in *NeurIPS*, 2024.
- [14] Z. Evans, J. D. Parker, C. Carr, Z. Zukowski, J. Taylor, and J. Pons, “Long-form music generation with latent diffusion,” in *ISMIR*, 2024, pp. 429–437.
- [15] Z. Ning, H. Chen, Y. Jiang, C. Hao, G. Ma, S. Wang, J. Yao, and L. Xie, “Diffrrhythm: Blazingly fast and embarrassingly simple end-to-end full-length song generation with latent diffusion,” *CoRR*, vol. abs/2503.01183, 2025.
- [16] Y. Chen, Z. Niu, Z. Ma, K. Deng, C. Wang, J. Zhao, K. Yu, and X. Chen, “F5-TTS: A fairytaler that fakes fluent and faithful speech with flow matching,” in *ACL (1)*. Association for Computational Linguistics, 2025, pp. 6255–6271.
- [17] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang, “Flow-grpo: Training flow matching models via online RL,” *CoRR*, vol. abs/2505.05470, 2025.
- [18] J. Li, Y. Cui, T. Huang, Y. Ma, C. Fan, M. Yang, and Z. Zhong, “Mixgrpo: Unlocking flow-based GRPO efficiency with mixed ODE-SDE,” *CoRR*, vol. abs/2507.21802, 2025.
- [19] H. Wang, B. Tian, Y. Jiang, Z. Pan, S. Zhao, B. Ma, D. Chen, and X. Li, “Flowse-grpo: Training flow matching speech enhancement via online reinforcement learning,” *CoRR*, vol. abs/2601.16483, 2026.
- [20] DeepSeek-AI, “Deepseek-v3.2: Pushing the frontier of open large language models,” *CoRR*, vol. abs/2512.02556, 2025.
- [21] H. He, Z. Shang, C. Wang, X. Li, Y. Gu, H. Hua, L. Liu, C. Yang, J. Li, P. Shi, Y. Wang, K. Chen, P. Zhang, and Z. Wu, “Emilia: An extensive, multilingual, and diverse speech dataset for large-scale speech generation,” in *SLT*. IEEE, 2024, pp. 885–890.
- [22] C. Hao, R. Yuan, J. Yao, Q. Deng, X. Bai, W. Xue, and L. Xie, “Songformer: Scaling music structure analysis with heterogeneous supervision,” *CoRR*, vol. abs/2510.02797, 2025.
- [23] J. Wang, W. T. Lu, and M. Won, “Mel-band roformer for music source separation,” *CoRR*, vol. abs/2310.01809, 2023.
- [24] A. Plaquet and H. Bredin, “Powerset multi-class cross entropy loss for neural speaker diarization,” in *INTERSPEECH*. ISCA, 2023, pp. 3222–3226.
- [25] H. Bredin, “pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe,” in *INTERSPEECH*. ISCA, 2023, pp. 1983–1987.
- [26] C. K. A. Reddy, V. Gopal, and R. Cutler, “Dnsmos: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors,” in *ICASSP*. IEEE, 2021, pp. 6493–6497.
- [27] —, “Dnsmos P.835: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors,” in *ICASSP*. IEEE, 2022, pp. 886–890.
- [28] X. Shi, X. Wang, Z. Guo, Y. Wang, P. Zhang, X. Zhang, Z. Guo, H. Hao, Y. Xi, B. Yang, J. Xu, J. Zhou, and J. Lin, “Qwen3-asr technical report,” *CoRR*, vol. abs/2601.21337, 2026.
- [29] H. Wei, X. Cao, T. Dan, and Y. Chen, “RMVPE: A robust model for vocal pitch estimation in polyphonic music,” in *INTERSPEECH*. ISCA, 2023, pp. 5421–5425.
- [30] Z. Wang, R. Yuan, Z. Geng, H. Li, X. Qu, X. Li, S. Chen, H. Fu, R. B. Dannenberg, and K. Zhang, “Singing timbre popularity assessment based on multimodal large foundation model,” in *ACM Multimedia*. ACM, 2025, pp. 12 227–12 236.